

Pandas para Análise de Dados: Guia Didático

Índice

1. [Criação de DataFrames](#)
 2. [Seleção de Dados](#)
 3. [Filtragem de Dados](#)
 4. [Agregações e Agrupamentos](#)
 5. [Manipulação de Elementos Específicos](#)
 6. [Limpeza de Dados](#)
 7. [Glossário de Funções](#)
-

Criação de DataFrames

Por que criar DataFrames?

Os DataFrames são a estrutura fundamental do Pandas - pense neles como tabelas inteligentes que permitem organizar, analisar e manipular dados de forma eficiente. Eles são essenciais porque:

- Organizam dados em formato tabular (linhas e colunas)
- Permitem operações vetorizadas (rápidas)
- Suportam diferentes tipos de dados em cada coluna
- São interoperáveis com diversas fontes de dados

Métodos de Criação: Escolhendo o Certo

Método 1: A partir de Dicionários (Mais Comum)

Quando usar: Quando seus dados já estão organizados em Python ou quando está criando dados de exemplo.

python

```
import pandas as pd
```

```
#  EXEMPLO: Criando um catálogo de produtos
```

```
# Cada chave do dicionário vira uma COLUNA do DataFrame
```

```
# Cada lista de valores vira os DADOS da coluna
```

```
dados_produtos = {
```

```
    'produto_id': [101, 102, 103, 104],      # Coluna de IDs
```

```
    'nome': ['Notebook', 'Mouse', 'Teclado', 'Monitor'], # Coluna de nomes
```

```
    'preco': [3500.00, 89.90, 149.90, 899.90], # Coluna de preços
```

```
    'estoque': [10, 50, 30, 15]              # Coluna de quantidades
```

```
}
```

```
# Transformando o dicionário em DataFrame
```

```
catalogo = pd.DataFrame(dados_produtos)
```

```
print("📦 Catálogo de Produtos Criado:")
```

```
print(catalogo)
```

```
print(f"\n📊 Formato: {catalogo.shape[0]} produtos x {catalogo.shape[1]}  
informações")
```

Método 2: A partir de Listas de Listas

Quando usar: Quando você recebe dados em formato tabular bruto (de APIs, scrapers, ou arquivos legados).

```
python
```

```
# 📝 EXEMPLO: Dados de vendas diárias
```

```
# Cada lista interna representa UMA LINHA de dados
```

```
# As colunas são definidas separadamente
```

```
vendas_diarias = [  
  
    # [data, vendedor, valor_venda, quantidade]  
  
    ['2023-10-01', 'Ana', 1500.00, 3],  
  
    ['2023-10-01', 'Bruno', 890.50, 2],  
  
    ['2023-10-02', 'Ana', 2200.00, 5],  
  
    ['2023-10-02', 'Carla', 1200.75, 4]  
  
]  
  
# Definindo os nomes das colunas (IMPORTANTE!)  
  
colunas = ['data', 'vendedor', 'valor_total', 'quantidade_itens']  
  
# Criando o DataFrame  
  
df_vendas = pd.DataFrame(vendas_diarias, columns=colunas)  
  
print("💰 Registro de Vendas Diárias:")
```

```
print(df_vendas)
```

Método 3: Carregando de Arquivos Externos (Mundo Real)

Quando usar: 90% do trabalho de um analista de dados começa aqui!

```
python
```

```
# 📁 CARREGANDO DE EXCEL (arquivos .xlsx, .xls)
```

```
# Excel é comum em ambientes corporativos
```

```
df_excel = pd.read_excel(
```

```
    'vendas_2023.xlsx',          # Nome do arquivo
```

```
    sheet_name='Vendas',        # Nome da planilha específica
```

```
    header=0,                    # Linha 0 contém os cabeçalhos
```

```
    dtype={'produto_id': str},   # Forçar tipo de dados específico
```

```
    parse_dates=['data_venda']  # Converter coluna para data automaticamente
```

```
)
```

```
print(f"✅ Excel carregado: {len(df_excel)} registros encontrados")
```

```
# 📄 CARREGANDO DE CSV (arquivos .csv)

# CSV é universal e compatível com quase tudo

df_csv = pd.read_csv(

    'clientes.csv',

    encoding='utf-8',      # Importante para acentos (use 'latin-1' se
                           # necessário)

    delimiter=',',        # Separador de colunas (pode ser ';' ou '\t')

    decimal='.',          # Separador decimal (',' ou '.')

    na_values=['NA', 'N/A', ''] # Valores que devem ser considerados como
                                # nulos

)

# 🔍 Verificando o que foi carregado

print("\n📋 Primeira inspeção dos dados:")

print(f"- Total de linhas: {df_csv.shape[0]}")

print(f"- Total de colunas: {df_csv.shape[1]}")

print(f"- Colunas disponíveis: {', '.join(df_csv.columns[:5])}...")
```

Boas Práticas na Criação

1. Sempre nomeie suas colunas claramente
 2. Verifique os tipos de dados após carregar
 3. Documente a origem dos dados
 4. Use encoding apropriado para textos em português
-

Seleção de Dados

Por que Selecionar Dados?

A seleção é como navegar em uma biblioteca de dados - você precisa encontrar exatamente as informações necessárias sem carregar tudo. É fundamental para:

- Reduzir o uso de memória
- Melhorar performance
- Focar na análise relevante
- Preparar dados para visualização

Técnicas de Seleção: Do Básico ao Avançado

Seleção Simples de Colunas

Pense assim: Você está em uma planilha e quer ver apenas algumas colunas.

```
python
```

 EXEMPLO: Sistema de RH - Selecionando informações específicas

```
dados_funcionarios = {  
  
    'matricula': [1001, 1002, 1003, 1004],  
  
    'nome': ['Ana Silva', 'Bruno Oliveira', 'Carla Santos', 'Diego Lima'],  
  
    'cargo': ['Analista', 'Gerente', 'Analista', 'Desenvolvedor'],  
  
    'salario': [5000.00, 8500.00, 5200.00, 6200.00],  
  
    'departamento': ['TI', 'Vendas', 'TI', 'TI'],  
  
    'data_admissao': ['2020-03-15', '2018-07-22', '2021-01-10', '2019-11-05']  
  
}  
  
df_rh = pd.DataFrame(dados_funcionarios)
```

 SELECIONANDO UMA COLUNA (retorna uma Series)

Use quando precisa trabalhar com apenas uma característica

```

salarios = df_rh['salario']

print("💰 Apenas salários (Series):")

print(type(salarios)) # <class 'pandas.core.series.Series'>

print(f"Média salarial: R$ {salarios.mean():.2f}")


# 🕒 SELECIONANDO MÚLTIPLAS COLUNAS (retorna DataFrame)

# Use quando precisa de um subconjunto de informações

info_basica = df_rh[['nome', 'cargo', 'departamento']]

print("\n👤 Informações básicas dos funcionários (DataFrame):")

print(info_basica)

print(f"Tipo: {type(info_basica)}") # <class 'pandas.core.frame.DataFrame'>

```

Seleção por Posição com `.iloc`

Pense assim: "Quero as primeiras 3 linhas e as colunas 1 e 3" - você está pensando em posições numéricas.

python

```
# 📊 EXEMPLO: Analisando amostras específicas por posição
```

```
# Primeiras 3 linhas, todas as colunas
```

```
amostra_inicial = df_rh.iloc[0:3] # Linhas 0, 1, 2
```

```
print("📋 Amostra inicial (3 primeiros funcionários):")
```

```
print(amostra_inicial)
```

```
# Linhas específicas, colunas específicas
```

```
# Formato: df.iloc[linhas, colunas]
```

```
dados_especificos = df_rh.iloc[[0, 2], [1, 3, 4]] # Linhas 0 e 2, colunas 1, 3 e 4
```

```
print("\n🎯 Dados específicos por posição numérica:")
```

```
print(dados_especificos)
```

```
# Última linha, todas as colunas
```

```
ultimo_funcionario = df_rh.iloc[-1] # Índice negativo conta do final
```

```
print("\n👤 Último funcionário cadastrado:")
```

```
print(ultimo_funcionario)
```

Seleção por Rótulo com `.loc`

Pense assim: "Quero os dados dos funcionários de TI" - você está pensando em rótulos/condições.

python

```
# 🌀 EXEMPLO: Selecionando por condições específicas
```

```
# Primeiro, vamos criar um índice mais significativo
```

```
df_rh_indexed = df_rh.set_index('matricula')
```

```
# Selecionando um funcionário específico pela matrícula
```

```
func_1002 = df_rh_indexed.loc[1002]
```

```
print("👤 Funcionário com matrícula 1002:")
```

```
print(func_1002)
```

```
# Selecionando múltiplos funcionários
```

```
funcs_ti = df_rh_indexed.loc[[1001, 1003, 1004]]
```

```
print("\n👤 Funcionários do departamento de TI:")
```

```
print(funcs_ti[['nome', 'cargo', 'salario']])
```

Seleção por Condições (Filtros Básicos)

Pense assim: "Mostre-me apenas os dados que atendem a certas condições"

python

```
# 🎯 EXEMPLO: Filtrando por características
```

```
# Criando uma "máscara" booleana (True/False para cada linha)
```

```
mask_ti = df_rh['departamento'] == 'TI'
```

```
print("👤 Máscara booleana para departamento TI:")
```

```
print(mask_ti)
```

```
# Aplicando a máscara para filtrar
```

```
funcionarios_ti = df_rh[mask_ti]
```

```
print("\n👤 Funcionários que trabalham em TI:")
```

```
print(funcionarios_ti)
```

```
# Condição complexa: TI com salário > 5000
```

```
mascara_complexa = (df_rh['departamento'] == 'TI') & (df_rh['salario'] > 5000)
```

```
funcionarios_ti_salario_alto = df_rh[mascara_complexa]
```

```
print("\n💡 Funcionários de TI com salário acima de R$ 5000:")
```

```
print(funcionarios_ti_salario_alto)
```

Quando Usar Cada Método?

Situação	Método Recomendado	Exemplo
Colunas específicas	Seleção direta	<code>df[['nome', 'idade']]</code>
Por posição numérica	<code>.iloc[]</code>	<code>df.iloc[0:5, 2:4]</code>
Por rótulos/índices	<code>.loc[]</code>	<code>df.loc[10:20, 'coluna']</code>

Por condições

Máscaras booleanas

`df[df['idade'] > 30]`

Erros Comuns e Como Evitá-los

python

❌ *ERRADO: Selecionar múltiplas colunas sem lista extra*

`nome_errado = df_rh['nome', 'cargo']` # *TypeError!*

✅ *CORRETO: Use lista dupla*

`nome_correto = df_rh[['nome', 'cargo']]`

❌ *ERRADO: Confundir .loc e .iloc*

`df.loc[0:2]` # *Procura por índices 0, 1, 2 (nomes, não posições)*

✅ *CORRETO: Use .iloc para posições*

`df.iloc[0:2]` # *Pega linhas nas posições 0 e 1*

Filtragem de Dados

Por que Filtrar Dados?

Filtrar é como peneirar areia para encontrar ouro - você remove o que não interessa para focar no que é valioso. É crucial para:

- Responder perguntas específicas de negócio
- Identificar padrões em subconjuntos
- Preparar dados para modelos específicos
- Criar visualizações focadas

Filtros Simples: O Ponto de Partida

Conceito: Filtros simples usam operadores de comparação básicos para isolar dados.

python

```
#  EXEMPLO: Sistema de vendas - filtrando por métricas
```

```
vendas = pd.DataFrame({  
  
    'vendedor': ['Ana', 'Bruno', 'Carla', 'Ana', 'Bruno', 'Carla'],  
  
    'valor': [1500, 800, 2200, 1700, 950, 1900],
```

```
        'produto': ['A', 'B', 'A', 'C', 'B', 'C'],  
  
        'comissao': [150, 80, 220, 170, 95, 190]  
  
    })
```

```
print("💰 Todas as vendas:")
```

```
print(vendas)
```

```
# 🎯 FILTRO DE MAIOR QUE (>)
```

```
# "Quais vendas foram acima de R$ 1000?"
```

```
vendas_grandes = vendas[vendas['valor'] > 1000]
```

```
print("\n🔥 Vendas acima de R$ 1000:")
```

```
print(vendas_grandes)
```

```
# 🎯 FILTRO DE MENOR QUE (<)
```

```
# "Quais vendas foram abaixo de R$ 1000?"
```

```
vendas_pequenas = vendas[vendas['valor'] < 1000]
```

```
print("\n💡 Vendas abaixo de R$ 1000:")
```

```
print(vendas_pequenas)
```

```
# 🎯 FILTRO DE IGUALDADE (==)
```

```
# "Quais vendas foram do produto A?"
```

```
vendas_produto_a = vendas[vendas['produto'] == 'A']
```

```
print("\n📦 Vendas do Produto A:")
```

```
print(vendas_produto_a)
```

Filtros Compostos: Combinando Condições

Conceito: Use operadores lógicos (& para AND, | para OR, ~ para NOT) para criar **filtros complexos**.

```
python
```

```
# 🎯 EXEMPLO: Análise de desempenho de vendedores
```

```
# 📌 REGRA: Parênteses são OBRIGATÓRIOS em condições múltiplas!
```

 AND (&): Ambas condições devem ser verdadeiras

"Vendas da Ana que foram acima de R\$ 1500"

```
vendas_ana_alta = vendas[(vendas['vendedor'] == 'Ana') & (vendas['valor'] > 1500)]
```

```
print("👤 Vendas da Ana acima de R$ 1500:")
```

```
print(vendas_ana_alta)
```

 OR (|): Pelo menos uma condição verdadeira

"Vendas do produto A OU vendas acima de R\$ 2000"

```
vendas_a_ou_alta = vendas[(vendas['produto'] == 'A') | (vendas['valor'] > 2000)]
```

```
print("\n📊 Vendas do produto A OU acima de R$ 2000:")
```

```
print(vendas_a_ou_alta)
```

 NOT (~): Inverte a condição

```
# "Vendas que NÃO são do produto B"
```

```
vendas_ao_b = vendas[(vendas['produto'] == 'B')]
```

```
print("\n🚫 Vendas que NÃO são do produto B:")
```

```
print(vendas_ao_b)
```

Filtros Avançados com `.query()`

Quando usar: Quando seus filtros ficam muito complexos ou você quer uma sintaxe mais limpa.

python

```
# 📌 EXEMPLO: Consultas complexas de vendas
```

```
# Sintaxe do .query(): escreva condições como strings
```

```
# Use @ para variáveis externas
```

```
# Filtro simples com .query()
```

```
vendas_ana = vendas.query('vendedor == "Ana"')
```

```
print("📝 Vendas da Ana (usando .query()):")
```

```
print(vendas_ana)
```

```
# Filtro composto com .query()
```

```
meta = 1000
```

```
vendas_acima_meta = vendas.query('valor > @meta and produto != "C"')
```

```
print(f"\n🎯 Vendas acima da meta (R$ {meta}) e que não são do produto C:")
```

```
print(vendas_acima_meta)
```

```
# Comparação: .query() vs filtro tradicional
```

```
print("\n⚖️ Comparação de sintaxes:")
```

```
print("Traditional: vendas[(vendas['valor'] > 1000) & (vendas['vendedor'] == 'Ana')]" )
```

```
print("Com .query(): vendas.query('valor > 1000 and vendedor == \"Ana\"')")
```

Filtros com `.isin()`: Múltiplos Valores

Quando usar: Quando precisa filtrar por vários valores específicos de uma vez.

```
python
```

```
# 📊 EXEMPLO: Relatório para gerentes específicos
```

```
# Lista de vendedores sob minha gestão
```

```
meus_vendedores = ['Ana', 'Carla']
```

```
# Filtro tradicional (verboso)
```

```
vendas_meus_vendedores_tradicional = vendas[
```

```
    (vendas['vendedor'] == 'Ana') | (vendas['vendedor'] == 'Carla')
```

```
]
```

```
# Filtro com .isin() (elegante)
```

```
vendas_meus_vendedores = vendas[vendas['vendedor'].isin(meus_vendedores)]
```

```
print("👥 Vendas dos meus vendedores (usando .isin()):")
```

```
print(vendas_meus_vendedores)
```

```
# 🎯 NEGAÇÃO com .isin()
```

```
# "Vendas de vendedores que NÃO estão na minha lista"
```

```
vendas_outros = vendas[~vendas['vendedor'].isin(meus_vendedores)]
```

```
print("\n👤 Vendas de outros vendedores:")
```

```
print(vendas_outros)
```

Filtros em Strings: Buscas Textuais

Quando usar: Quando precisa encontrar padrões em textos (nomes, emails, descrições).

```
python
```

```
# 📝 EXEMPLO: Sistema de clientes com buscas textuais
```

```
clientes = pd.DataFrame({
```

```
    'nome': ['Ana Silva', 'Bruno Oliveira Santos', 'Carla Pereira', 'Diego  
Silva Costa'],
```

```
    'email': ['ana.silva@empresa.com', 'bruno.os@outro.com',  
'carla@startup.com', 'diego.sc@gmail.com'],
```

```
        'cidade': ['São Paulo', 'Rio de Janeiro', 'São Paulo', 'Porto Alegre']

    })
```

```
# 🔍 CONTAINS: Texto que contém padrão
```

```
# "Clientes com 'Silva' no nome"
```

```
clientes_silva = clientes[clientes['nome'].str.contains('Silva')]
```

```
print(f"🔍 Clientes com 'Silva' no nome:")
```

```
print(clientes_silva)
```

```
# 🔍 STARTSWITH: Texto que começa com padrão
```

```
# "Clientes com email da empresa"
```

```
clientes_empresa = clientes[clientes['email'].str.startswith('ana')]
```

```
print(f"\n📋 Clientes com email começando com 'ana':")
```

```
print(clientes_empresa)
```

```
# 🎯 ENDSWITH: Texto que termina com padrão
```

```
# "Clientes com email .com"
```

```
clientes_com = clientes[clientes['email'].str.endswith('.com')]
```

```
print("\n🏠 Clientes com email .com:")
```

```
print(clientes_com)
```

```
# 🎯 IGNORECASE: Ignorar maiúsculas/minúsculas
```

```
clientes_sao = clientes[clientes['cidade'].str.contains('são', case=False)]
```

```
print("\n🏠 Clientes de São Paulo (case insensitive):")
```

```
print(clientes_sao)
```

Caso Prático: Análise de Vendas com Filtros

```
python
```

```
# 🎯 DESAFIO: Criar relatório gerencial com múltiplos filtros
```

```
# 1. Vendas acima da média
```

```
media_vendas = vendas['valor'].mean()
```

```
vendas_acima_media = vendas[vendas['valor'] > media_vendas]
```

```
# 2. Vendedores com mais de 1 venda
```

```
contagem_vendedores = vendas['vendedor'].value_counts()
```

```
vendedores_ativos = contagem_vendedores[contagem_vendedores > 1].index
```

```
vendas_vendedores_ativos = vendas[vendas['vendedor'].isin(vendedores_ativos)]
```

```
print("📊 RELATÓRIO GERENCIAL:")
```

```
print(f"1. Média de vendas: R$ {media_vendas:.2f}")
```

```
print(f"2. Vendas acima da média: {len(vendas_acima_media)} transações")
```

```
print(f"3. Vendedores ativos (mais de 1 venda): {'',  
'.'.join(vendedores_ativos)}")
```

```
print(f"4. Vendas dos vendedores ativos: {len(vendas_vendedores_ativos)}  
transações")
```

Dicas Profissionais de Filtragem

1. Teste filtros complexos por partes
2. Use variáveis intermediárias para clareza
3. Documente a lógica de filtros importantes

4. Valide resultados com contagens e estatísticas
 5. Crie funções reutilizáveis para filtros comuns
-

Agregações e Agrupamentos

Por que Agregar e Agrupar?

Agregações transformam dados brutos em insights - são como resumir um livro de 500 páginas em um relatório executivo de 10 páginas. Essenciais para:

- Identificar padrões e tendências
- Comparar diferentes grupos
- Criar métricas de desempenho
- Preparar dados para tomada de decisão

Funções de Agregação Básicas: Seu Kit de Ferramentas

```
python
```

```
#  EXEMPLO: Análise de desempenho de lojas
```

```
vendas_lojas = pd.DataFrame({  
  
    'loja': ['Norte', 'Sul', 'Norte', 'Sul', 'Norte', 'Sul', 'Centro',  
    'Centro'],
```

```
        'vendedor': ['Ana', 'Bruno', 'Carla', 'Diego', 'Ana', 'Bruno', 'Elena',  
        'Fernando'],  
  
        'valor': [1500, 1200, 1800, 900, 2000, 1100, 1700, 1300],  
  
        'clientes': [3, 2, 4, 1, 5, 2, 3, 2]  
  
    })
```

```
print("📊 Dados completos de vendas por loja:")
```

```
print(vendas_lojas)
```

```
# 🎯 SUM: Soma total
```

```
# "Qual o valor total vendido?"
```

```
total_vendas = vendas_lojas['valor'].sum()
```

```
print(f"\n💰 TOTAL DE VENDAS: R$ {total_vendas:.2f}")
```

```
# 🎯 MEAN: Média aritmética
```

```
# "Qual o valor médio por venda?"
```

```
media_venda = vendas_lojas['valor'].mean()
```

```
print(f"📊 MÉDIA POR VENDA: R$ {media_venda:.2f}")
```

```
# 🎯 MEDIAN: Mediana (valor do meio)
```

```
# "Qual o valor típico, ignorando extremos?"
```

```
mediana_venda = vendas_lojas['valor'].median()
```

```
print(f"🎯 MEDIANA: R$ {mediana_venda:.2f}")
```

```
# 🎯 MIN/MAX: Valores extremos
```

```
# "Qual foi a menor/menor venda?"
```

```
min_venda = vendas_lojas['valor'].min()
```

```
max_venda = vendas_lojas['valor'].max()
```

```
print(f"📉 MENOR VENDA: R$ {min_venda:.2f}")
```

```
print(f"📈 MAIOR VENDA: R$ {max_venda:.2f}")
```

```
# 🎯 COUNT: Contagem
```

```
# "Quantas vendas foram registradas?"
```

```
total_vendas_count = vendas_lojas['valor'].count()
```

```
print(f"📊 TOTAL DE TRANSAÇÕES: {total_vendas_count}")
```

```
# 🎯 STD: Desvio padrão
```

```
# "Quão variáveis são as vendas?"
```

```
desvio_venda = vendas_lojas['valor'].std()
```

```
print(f"📊 DESVIO PADRÃO: R$ {desvio_venda:.2f} (medida de variabilidade)")
```

Agrupamentos: A Magia do `.groupby()`

Conceito: Dividir dados em grupos, aplicar função a cada grupo, combinar resultados.

```
python
```

```
# 🎯 EXEMPLO: Comparando desempenho por loja
```

```
# Passo 1: Agrupar por loja
```

```
grupo_lojas = vendas_lojas.groupby('loja')
```

```
print("👥 Grupos criados:")
```

```
print(f"- Número de grupos: {grupo_lojas.ngroups}")
```

```
print(f"- Nomes dos grupos: {grupo_lojas.groups.keys()}")
```

```
# Passo 2: Aplicar agregações a cada grupo
```

```
# Método 1: Agregação simples por coluna
```

```
total_por_loja = vendas_lojas.groupby('loja')['valor'].sum()
```

```
print("\n💰 TOTAL DE VENDAS POR LOJA:")
```

```
print(total_por_loja)
```

```
# Método 2: Múltiplas agregações em uma coluna
```

```
estatisticas_por_loja = vendas_lojas.groupby('loja')['valor'].agg(['sum',  
'mean', 'count', 'std'])
```

```
print("\n📊 ESTATÍSTICAS DETALHADAS POR LOJA:")
```

```
print(estatisticas_por_loja)
```

```
# Método 3: Agregações diferentes por colunas diferentes
```

```
resumo_completo = vendas_lojas.groupby('loja').agg({  
  
    'valor': ['sum', 'mean', 'max'], # Para coluna 'valor'  
  
    'clientes': 'sum', # Para coluna 'clientes'  
  
    'vendedor': 'nunique' # Contar vendedores únicos  
  
})
```

```
print("\n🎯 RESUMO COMPLETO POR LOJA:")
```

```
print(resumo_completo)
```

Transformando Resultados de Agrupamento

```
python
```

```
# 🎯 EXEMPLO: Análise de participação de mercado
```

```
# Calculando totais e porcentagens
```

```
vendas_por_loja = vendas_lojas.groupby('loja').agg({

    'valor': 'sum',

    'clientes': 'sum'

}).reset_index()


# Adicionando porcentagem do total

total_geral = vendas_por_loja['valor'].sum()

vendas_por_loja['participacao'] = (vendas_por_loja['valor'] / total_geral *
100).round(2)


# Adicionando ticket médio (valor por cliente)

vendas_por_loja['ticket_medio'] = (vendas_por_loja['valor'] /
vendas_por_loja['clientes']).round(2)


print("📊 ANÁLISE DE PARTICIPAÇÃO DE MERCADO:")

print(vendas_por_loja.sort_values('participacao', ascending=False))
```

Agrupamentos Múltiplos: Níveis Hierárquicos

Quando usar: Quando precisa analisar dados por mais de uma dimensão.

python

🎯 EXEMPLO: Vendas por loja E por vendedor

Agrupando por loja E vendedor

```
agrupamento_duplo = vendas_lojas.groupby(['loja', 'vendedor']).agg({
```

```
    'valor': 'sum',
```

```
    'clientes': 'sum'
```

```
}).reset_index()
```

```
print("👥 VENDAS POR LOJA E VENDEDOR:")
```

```
print(agrupamento_duplo)
```

🎯 Adicionando cálculo de performance relativa

Ticket médio por vendedor

```
agrupamento_duplo['ticket_medio'] = (agrupamento_duplo['valor'] /
agrupamento_duplo['clientes']).round(2)
```

```
# Performance vs média da loja
```

```
for loja in agrupamento_duplo['loja'].unique():
```

```
    media_loja = agrupamento_duplo[agrupamento_duplo['loja'] ==
loja]['ticket_medio'].mean()
```

```
    mascara = agrupamento_duplo['loja'] == loja
```

```
    agrupamento_duplo.loc[mascara, 'vs_media_loja'] = (
```

```
        agrupamento_duplo.loc[mascara, 'ticket_medio'] / media_loja * 100

    ).round(2)
```

```
print("\n📊 PERFORMANCE DETALHADA:")
```

```
print(agrupamento_duplo.sort_values(['loja', 'valor'], ascending=[True,
False]))
```

Tabelas Dinâmicas com `.pivot_table()`

Quando usar: Quando precisa de uma visualização tabular cruzada (como no Excel).

python

🎯 EXEMPLO: Criando relatório executivo estilo Excel

```
tabela_dinamica = pd.pivot_table(

    vendas_lojas,

    values='valor',          # Valores a agregar

    index='loja',           # Linhas da tabela

    columns='vendedor',     # Colunas da tabela

    aggfunc='sum',          # Função de agregação

    fill_value=0,           # Preencher valores faltantes

    margins=True,           # Adicionar totais

    margins_name='TOTAL'    # Nome da linha/coluna de totais

)
```

```
print("📊 TABELA DINÂMICA - VENDAS POR LOJA E VENDEDOR:")
```

```
print(tabela_dinamica)
```

```
# 🎯 Tabela dinâmica com múltiplas métricas
```

```
tabela_completa = pd.pivot_table(  
  
    vendas_lojas,  
  
    values=['valor', 'clientes'],  
  
    index='loja',  
  
    aggfunc={'valor': ['sum', 'mean'], 'clientes': 'sum'},  
  
    margins=True  
  
)
```

```
print("\n📈 TABELA COMPLETA COM MÚLTIPLAS MÉTRICAS:")
```

```
print(tabela_completa)
```

Análise de Frequência com `.value_counts()`

Quando usar: Para entender distribuição de categorias.

python

```
# 🎯 EXEMPLO: Análise de distribuição de vendas
```

```
# Contagem simples
```

```
frequencia_vendedores = vendas_lojas['vendedor'].value_counts()
```

```
print("👤 FREQUÊNCIA DE VENDAS POR VENDEDOR:")
```

```
print(frequencia_vendedores)
```

```
# Com porcentagens
```

```
frequencia_percentual = vendas_lojas['vendedor'].value_counts(normalize=True)
```

```
* 100
```

```
print("\n📊 DISTRIBUIÇÃO PERCENTUAL:")
```

```
print(frequencia_percentual.round(2))
```

```
# Com ordenação personalizada
```

```
print("\n🎯 TOP 3 VENDEDORES:")
```

```
top_vendedores = frequencia_vendedores.head(3)
```

```
for i, (vendedor, vendas_count) in enumerate(top_vendedores.items(), 1):
```

```
    print(f"{i}º: {vendedor} - {vendas_count} vendas")
```

Caso Prático: Dashboard de Performance

python

```
# 🎯 DESAFIO: Criar dashboard de performance mensal
```

```
# Dados mensais
```

```
vendas_mensais = pd.DataFrame({
```

```
    'mes': ['Jan', 'Jan', 'Feb', 'Feb', 'Mar', 'Mar', 'Apr', 'Apr'],
```

```
    'categoria': ['Eletrônicos', 'Móveis', 'Eletrônicos', 'Móveis',  
'Eletrônicos', 'Móveis', 'Eletrônicos', 'Móveis'],
```

```
    'vendas': [50000, 30000, 55000, 32000, 60000, 35000, 52000, 38000],
```

```
    'lucro': [15000, 9000, 16500, 9600, 18000, 10500, 15600, 11400]
```

```
})
```

```
print("📊 DASHBOARD DE PERFORMANCE - ANÁLISE MENSAL")
```

```
print("=" * 50)
```

```
# 1. Performance total por mês
```

```
performance_mensal = vendas_mensais.groupby('mes').agg({
```

```
    'vendas': 'sum',
```

```
    'lucro': 'sum'
```

```
})
```

```
performance_mensal['margem'] = (performance_mensal['lucro'] /  
performance_mensal['vendas'] * 100).round(2)
```

```
print("\n1. 📅 PERFORMANCE POR MÊS:")
```

```
print(performance_mensal)
```

```
# 2. Comparativo por categoria
```

```
performance_categoria = vendas_mensais.groupby('categoria').agg({

    'vendas': 'sum',

    'lucro': 'sum'

})

performance_categoria['participacao'] = (

    performance_categoria['vendas'] / performance_categoria['vendas'].sum() *

100

).round(2)

print("\n2. 📊 PERFORMANCE POR CATEGORIA:")

print(performance_categoria)
```

3. Crescimento mês a mês

```
crescimento = performance_mensal['vendas'].pct_change() * 100

print("\n3. 📈 CRESCIMENTO MÊS A MÊS:")

for mes, cres in crescimento.items():
```

```
if pd.isna(cres):
```

```
    print(f"{mes}: {cres:.2f}%")
```

Boas Práticas em Agregações

1. Sempre nomeie colunas resultantes claramente
 2. Documente a lógica de cada agregação
 3. Valide resultados com sanity checks
 4. Use `.reset_index()` quando precisar do DataFrame plano
 5. Considere performance com datasets grandes
-

Manipulação de Elementos Específicos

Por que Manipular Elementos Específicos?

Às vezes você precisa fazer cirurgia nos dados - modificar uma célula, ajustar uma linha, ou transformar uma coluna específica. Essas operações são essenciais para:

- Corrigir erros pontuais
- Padronizar formatos
- Preparar dados para integração
- Ajustar outliers específicos

Trabalhando com Linhas: Adicionando e Removendo

python

```
# 📊 EXEMPLO: Sistema de inventário em tempo real
```

```
inventario = pd.DataFrame({  
  
    'produto_id': ['P001', 'P002', 'P003'],  
  
    'produto': ['Notebook', 'Mouse', 'Teclado'],  
  
    'estoque': [15, 42, 28],  
  
    'preco': [3500.00, 89.90, 149.90]  
  
})
```

```
print("📦 INVENTÁRIO ATUAL:")
```

```
print(inventario)
```

```
# 🎯 ADICIONANDO UMA NOVA LINHA (produto novo)
```

```
# Método 1: Usando dicionário (recomendado)
```

```
novo_produto = pd.DataFrame({
```

```
'produto_id': ['P004'],

'produto': ['Monitor'],

'estoque': [8],

'preco': [899.90]

})

# IMPORTANTE: ignore_index=True para manter índice limpo

inventario_atualizado = pd.concat([inventario, novo_produto],
ignore_index=True)

print("\n✅ INVENTÁRIO COM NOVO PRODUTO ADICIONADO:")

print(inventario_atualizado)


# 🚫 REMOVENDO UMA LINHA (produto descontinuado)

# Remover produto P002 (Mouse)

inventario_final = inventario_atualizado.drop(index=1) # Remove linha de
índice 1
```

```
print("\n❌ INVENTÁRIO APÓS REMOVER PRODUTO P002:")
```

```
print(inventario_final)
```

```
# 🎯 REMOVENDO MÚLTIPLAS LINHAS
```

```
# Remover produtos com baixo estoque (< 10)
```

```
mascara_baixo_estoque = inventario_final['estoque'] < 10
```

```
produtos_baixo_estoque = inventario_final[mascara_baixo_estoque]
```

```
inventario_limpo = inventario_final[~mascara_baixo_estoque]
```

```
print(f"\n⚠️ PRODUTOS COM BAIXO ESTOQUE REMOVIDOS  
( {len(produtos_baixo_estoque)} itens):")
```

```
print(produtos_baixo_estoque[['produto', 'estoque']])
```

Trabalhando com Colunas: Transformando Dados

```
python
```

```
# 📝 EXEMPLO: Sistema de RH com transformações de dados
```

```
rh_data = pd.DataFrame({

    'funcionario_id': [101, 102, 103, 104],

    'nome_completo': ['ana silva', 'BRUNO OLIVEIRA', 'Carla Santos', 'diego
lima'],

    'salario_base': [5000, 6000, 5500, 4800],

    'horas_extras': [10, 5, 8, 12],

    'data_admissao': ['2020-03-15', '2019-07-22', '2021-01-10', '2018-11-05']

})
```

```
print("👤 DADOS BRUTOS DO RH:")
```

```
print(rh_data)
```

```
# 🔄 TRANSFORMAÇÃO 1: Padronizar nomes (Title Case)
```

```
rh_data['nome_formatado'] = rh_data['nome_completo'].str.title()
```

```
print("\n✨ NOMES PADRONIZADOS:")
```

```
print(rh_data[['nome_completo', 'nome_formatado']])
```

```
# 🔄 TRANSFORMAÇÃO 2: Calcular salário total
```

```
# Supondo: hora extra = 1.5 × salário hora
```

```
# Salário hora = salário_base / 160 (horas mês)
```

```
rh_data['salario_hora'] = rh_data['salario_base'] / 160
```

```
rh_data['valor_horas_extras'] = rh_data['horas_extras'] *  
rh_data['salario_hora'] * 1.5
```

```
rh_data['salario_total'] = rh_data['salario_base'] +  
rh_data['valor_horas_extras']
```

```
print("\n💰 CÁLCULO DE SALÁRIO TOTAL:")
```

```
print(rh_data[['nome_formatado', 'salario_base', 'horas_extras',  
'salario_total']]).round(2))
```

```
# 🔄 TRANSFORMAÇÃO 3: Criar categorias
```

```
# Classificar funcionários por tempo de empresa
```

```
from datetime import datetime
```

```
rh_data['data_admissao'] = pd.to_datetime(rh_data['data_admissao'])

hoje = datetime.now()

rh_data['anos_empresa'] = ((hoje - rh_data['data_admissao']).dt.days /
365).round(1)


# Criar categorias baseadas em anos

categorias = pd.cut(

    rh_data['anos_empresa'],

    bins=[0, 1, 3, 10],

    labels=['Novato', 'Experiente', 'Veterano']

)

rh_data['categoria_experiencia'] = categorias


print("\n🏆 CATEGORIA POR EXPERIÊNCIA:")

print(rh_data[['nome_formatado', 'anos_empresa', 'categoria_experiencia']])
```

Modificando Células Específicas

python

🎯 EXEMPLO: Correção de dados pontuais

Dados com erros

```
vendas_erradas = pd.DataFrame({

    'venda_id': [1, 2, 3, 4, 5],

    'produto': ['A', 'B', 'A', 'C', 'B'],

    'quantidade': [10, -5, 15, 8, 12], # Erro: quantidade negativa

    'valor': [1000, 500, 1500, 800, None] # Erro: valor faltante

})

print("📝 DADOS COM ERROS IDENTIFICADOS:")

print(vendas_erradas)
```

```
# 🎯 CORREÇÃO 1: Substituir valor específico
```

```
# Correção: quantidade negativa deve ser positiva
```

```
vendas_erradas.loc[1, 'quantidade'] = 5 # Linha 1, coluna 'quantidade'
```

```
print("\n✅ CORREÇÃO 1: Quantidade negativa corrigida")
```

```
# 🎯 CORREÇÃO 2: Preencher valor faltante com média
```

```
media_valor = vendas_erradas['valor'].mean()
```

```
vendas_erradas.loc[4, 'valor'] = media_valor # Linha 4, coluna 'valor'
```

```
print(f"✅ CORREÇÃO 2: Valor faltante preenchido com média (R$  
{media_valor:.2f})")
```

```
# 🎯 CORREÇÃO 3: Aplicar regra de negócio
```

```
# Se quantidade > 10, aplicar desconto de 5%
```

```
mascara_grande_pedido = vendas_erradas['quantidade'] > 10
```

```
vendas_erradas.loc[mascara_grande_pedido, 'valor'] =  
vendas_erradas.loc[mascara_grande_pedido, 'valor'] * 0.95
```

```
print("\n🎯 DADOS CORRIGIDOS E APLICADAS REGRAS DE NEGÓCIO:")
```

```
print(vendas_erradas)
```

Ordenação e Reorganização

python

```
# 📊 EXEMPLO: Ranking de produtos por desempenho
```

```
desempenho_produtos = pd.DataFrame({

    'produto': ['Notebook', 'Mouse', 'Teclado', 'Monitor', 'Webcam'],

    'vendas': [150, 300, 200, 80, 120],

    'lucro': [45000, 9000, 20000, 16000, 6000],

    'satisfacao': [4.5, 4.2, 4.7, 4.8, 4.3]

})
```

```
print("📈 DESEMPENHO BRUTO DOS PRODUTOS:")
```

```
print(desempenho_produtos)
```

```
# 🎯 ORDENAÇÃO 1: Por uma coluna (vendas descendente)
```

```
produtos_mais_vendidos = desempenho_produtos.sort_values('vendas',  
ascending=False)
```

```
print("\n🏆 PRODUTOS MAIS VENDIDOS:")
```

```
print(produtos_mais_vendidos[['produto', 'vendas']])
```

```
# 🎯 ORDENAÇÃO 2: Por múltiplas colunas
```

```
# Prioridade: satisfação (descendente), depois lucro (descendente)
```

```
produtos_balanceados = desempenho_produtos.sort_values(
```

```
    ['satisfacao', 'lucro'],
```

```
    ascending=[False, False]
```

```
)
```

```
print("\n⚖️ PRODUTOS BALANCEADOS (SATISFAÇÃO + LUCRO):")
```

```
print(produtos_balanceados[['produto', 'satisfacao', 'lucro']])
```

```
# 🎯 ORDENAÇÃO 3: Com ranking
```

```
desempenho_produtos['rank_vendas'] =  
desempenho_produtos['vendas'].rank(ascending=False, method='dense')  
  
desempenho_produtos['rank_lucro'] =  
desempenho_produtos['lucro'].rank(ascending=False, method='dense')  
  
desempenho_produtos['rank_geral'] = (  
  
    desempenho_produtos['rank_vendas'] + desempenho_produtos['rank_lucro']  
  
) / 2  
  
print("\n🏆 RANKING COMPLETO DOS PRODUTOS:")  
print(desempenho_produtos.sort_values('rank_geral'))
```

Transformações com `.apply()`

python

```
# 🎯 EXEMPLO: Sistema de classificação de clientes
```

```
clientes = pd.DataFrame({  
  
    'cliente_id': [1, 2, 3, 4, 5],
```

```
'nome': ['Ana', 'Bruno', 'Carla', 'Diego', 'Elena'],

'compras_12meses': [8, 15, 3, 20, 5],

'ticket_medio': [150.00, 200.00, 300.00, 180.00, 250.00],

'ultima_compra_dias': [10, 5, 60, 2, 30]

})
```

```
print("👤 DADOS BRUTOS DOS CLIENTES:")
```

```
print(clientes)
```

```
# 🎯 FUNÇÃO PERSONALIZADA para classificação
```

```
def classificar_cliente(linha):
```

```
    """Classifica cliente baseado em múltiplos fatores"""
```

```
    score = 0
```

```
# Fator 1: Frequência de compras
```

```
if linha['compras_12meses'] > 10:
```

```
    score += 3
```

```
elif linha['compras_12meses'] > 5:
```

```
    score += 2
```

```
else:
```

```
    score += 1
```

```
# Fator 2: Ticket médio
```

```
if linha['ticket_medio'] > 250:
```

```
    score += 3
```

```
elif linha['ticket_medio'] > 150:
```

```
    score += 2
```

```
else:
```

```
    score += 1
```

```
# Fator 3: Recência
```

```
if linha['ultima_compra_dias'] < 7:
```

```
    score += 3
```

```
elif linha['ultima_compra_dias'] < 30:
```

```
    score += 2
```

```
else:
```

```
    score += 1
```

```
# Classificação final
```

```
if score >= 8:
```

```
    return 'A - Premium'
```

```
elif score >= 6:
```

```
    return 'B - Regular'
```

```
else:
```

```
    return 'C - Ocasional'
```

```
# 🎯 APLICANDO FUNÇÃO A CADA LINHA
```

```
clientes['classificacao'] = clientes.apply(classificar_cliente, axis=1)
```

```
print("\n🎯 CLIENTES CLASSIFICADOS:")
```

```
print(clientes[['nome', 'compras_12meses', 'ticket_medio', 'classificacao']])
```

Técnicas Avançadas de Manipulação

python

```
# 📊 EXEMPLO: Análise de séries temporais com transformações
```

```
# Dados de vendas diárias
```

```
vendas_diarias = pd.DataFrame({
```

```
    'data': pd.date_range('2023-10-01', periods=10, freq='D'),
```

```
    'vendas': [1000, 1200, 800, 1500, 900, 1300, 1100, 1400, 1000, 1600]
```

```
})
```

```
print("📅 VENDAS DIÁRIAS:")
```

```
print(vendas_diarias)
```

```
# 🎯 TRANSFORMAÇÃO 1: Média móvel (suaviza variações)
```

```
vendas_diarias['media_movel_3d'] =  
vendas_diarias['vendas'].rolling(window=3).mean()
```

```
print("\n📊 MÉDIA MÓVEL DE 3 DIAS:")
```

```
print(vendas_diarias[['data', 'vendas', 'media_movel_3d']])
```

```
# 🎯 TRANSFORMAÇÃO 2: Diferença dia anterior
```

```
vendas_diarias['diff_dia_anterior'] = vendas_diarias['vendas'].diff()
```

```
print("\n📈 VARIAÇÃO DIÁRIA:")
```

```
print(vendas_diarias[['data', 'vendas', 'diff_dia_anterior']])
```

```
# 🎯 TRANSFORMAÇÃO 3: Acumulado do mês
```

```
vendas_diarias['acumulado_mes'] = vendas_diarias['vendas'].cumsum()
```

```
print("\n💰 ACUMULADO DO MÊS:")
```

```
print(vendas_diarias[['data', 'vendas', 'acumulado_mes']])
```

```
# 🔄 TRANSFORMAÇÃO 4: Crescimento percentual
```

```
vendas_diarias['crescimento_pct'] = vendas_diarias['vendas'].pct_change() *  
100
```

```
print("\n📊 CRESCIMENTO PERCENTUAL:")
```

```
print(vendas_diarias[['data', 'vendas', 'crescimento_pct']].round(2))
```

Caso Prático: Pipeline de Transformação

```
python
```

```
# 🔄 DESAFIO: Pipeline completo de preparação de dados
```

```
def pipeline_transformacao(df):
```

```
    """
```

```
    Pipeline completo para transformar dados brutos em dados analíticos
```

```
"""
```

```
# 1. Cópia para não modificar original
```

```
df_transformado = df.copy()
```

```
print("🔧 INICIANDO PIPELINE DE TRANSFORMAÇÃO")
```

```
print(f"  Entrada: {df_transformado.shape[0]} linhas ×  
{df_transformado.shape[1]} colunas")
```

```
# 2. Correção de tipos de dados
```

```
if 'data' in df_transformado.columns:
```

```
    df_transformado['data'] = pd.to_datetime(df_transformado['data'])
```

```
    print("  ✅ Datas convertidas")
```

```
# 3. Padronização de textos
```

```
colunas_texto = df_transformado.select_dtypes(include=['object']).columns
```

```
for col in colunas_texto:
```

```
    df_transformado[col] = df_transformado[col].str.strip().str.title()
```

```
print(f"    ✅ {len(colunas_texto)} colunas de texto padronizadas")
```

```
# 4. Cálculo de métricas derivadas
```

```
if all(col in df_transformado.columns for col in ['quantidade',  
'preco_unitario']):
```

```
    df_transformado['valor_total'] = df_transformado['quantidade'] *  
df_transformado['preco_unitario']
```

```
print("    ✅ Valor total calculado")
```

```
# 5. Categorização
```

```
if 'valor_total' in df_transformado.columns:
```

```
    bins = [0, 100, 500, 1000, float('inf')]
```

```
    labels = ['Pequeno', 'Médio', 'Grande', 'Muito Grande']
```

```
    df_transformado['tamanho_venda'] =  
pd.cut(df_transformado['valor_total'], bins=bins, labels=labels)
```

```
print("    ✓ Vendas categorizadas por tamanho")
```

```
# 6. Ordenação final
```

```
if 'data' in df_transformado.columns:
```

```
    df_transformado = df_transformado.sort_values('data')
```

```
print("    ✓ Dados ordenados por data")
```

```
print(f"    🎯 Saída: {df_transformado.shape[0]} linhas x  
{df_transformado.shape[1]} colunas")
```

```
return df_transformado
```

```
# Testando o pipeline
```

```
dados_brutos = pd.DataFrame({
```

```
    'data': ['2023-10-01', '2023-10-02', '2023-10-03'],
```

```
    'cliente': ['ana silva ', 'BRUNO OLIVEIRA', 'carla santos '],
```

```
'quantidade': [2, 1, 3],  
  
'preco_unitario': [500.00, 300.00, 200.00]  
  
})
```

```
dados_processados = pipeline_transformacao(dados_brutos)
```

```
print("\n🔗 DADOS PROCESSADOS:")
```

```
print(dados_processados)
```

Melhores Práticas para Manipulação

1. Sempre faça backup antes de manipulações destrutivas
2. Use `.copy()` quando criar novas DataFrames
3. Documente transformações complexas
4. Valide resultados após cada transformação importante
5. Crie funções reutilizáveis para transformações comuns



Limpeza de Dados

Por que Limpar Dados?

Dados limpos são como matéria-prima de qualidade - essenciais para análises confiáveis. Estima-se que analistas gastam 60-80% do tempo limpando dados porque:

- Dados reais são "sujos" por natureza
- Múltiplas fontes criam inconsistências
- Erros humanos são comuns
- Sistemas legados têm formatos diferentes

Identificando Problemas Comuns

python

 EXEMPLO: Dados "sujos" típicos do mundo real

```
dados_sujos = pd.DataFrame({

    'cliente_id': [1, 2, 3, 4, 5, 6],

    'nome': ['Ana', 'Bruno', None, 'Diego', 'Elena', 'Fabio'],

    'idade': [25, 32, 28, 'quarenta', 36, 22], # Mix de tipos

    'email': ['ana@email.com', 'bruno@email', 'carla@email.com',

              None, 'elena@email.com', 'fabio@email.com'],

    'telefone': ['(11) 99999-9999', '11988888888', None,
```

```

        '(21) 97777-7777', '31966666666', '' ],

        'valor_compra': [100.50, 200.00, 150.75, 300.00, None, 180.25],

        'data_compra': ['2023-10-01', '2023-10-02', '2023/10/03',

                        '01-10-2023', '2023-10-05', '2023-10-06']

    })

```

```

print("🗑 DADOS COM PROBLEMAS IDENTIFICADOS:")

```

```

print(dados_sujos)

```

```

print("\n🔍 RESUMO DOS PROBLEMAS:")

```

```

print(dados_sujos.info())

```

Detecção Sistemática de Problemas

```
python
```

```

def diagnosticar_dados(df, nome_dataset="Dataset"):

```

```

    """

```

```

    Função para diagnóstico completo de problemas nos dados

```

```

    """

```

```
print(f"\n{'='*60}")
```

```
print(f"🔍 DIAGNÓSTICO: {nome_dataset}")
```

```
print(f"{'='*60}")
```

```
# 1. Dimensões
```

```
print(f"\n📊 DIMENSÕES: {df.shape[0]} linhas × {df.shape[1]} colunas")
```

```
# 2. Valores nulos
```

```
print("\n? VALORES NULOS POR COLUNA:")
```

```
nulos = df.isnull().sum()
```

```
for col, count in nulos.items():
```

```
    if count > 0:
```

```
        pct = (count / len(df)) * 100
```

```
        print(f"    {col}: {count} nulos ({pct:.1f}%)")
```

3. Tipos de dados

```
print("\n📄 TIPOS DE DADOS:")
```

```
for col, dtype in df.dtypes.items():
```

```
    print(f"    {col}: {dtype}")
```

4. Valores únicos

```
print("\n🎯 VALORES ÚNICOS (TOP 5):")
```

```
for col in df.columns:
```

```
    if df[col].nunique() < 10: # Mostrar apenas colunas com poucos  
valores únicos
```

```
        print(f"    {col}: {df[col].unique()[:5]}")
```

5. Estatísticas básicas para numéricas

```
print("\n📊 ESTATÍSTICAS PARA COLUNAS NUMÉRICAS:")
```

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
if len(numeric_cols) > 0:

    print(df[numeric_cols].describe().round(2))


return nulos
```

```
# Executando diagnóstico
```

```
problemas = diagnosticar_dados(dados_sujos, "Vendas de Clientes")
```

Estratégias de Limpeza: Um Processo Passo a Passo

```
python
```

```
def limpar_dados_completamente(df):

    """

    Pipeline completo de limpeza de dados

    """

    print("🧹 INICIANDO LIMPEZA COMPLETA DE DADOS")
```

PASSO 1: Cópia de segurança

```
df_limpo = df.copy()
```

```
print("✅ Passo 1: Cópia de segurança criada")
```

PASSO 2: Remover espaços em branco

```
colunas_texto = df_limpo.select_dtypes(include=['object']).columns
```

```
for col in colunas_texto:
```

```
    df_limpo[col] = df_limpo[col].astype(str).str.strip()
```

```
print(f"✅ Passo 2: Espaços removidos de {len(colunas_texto)} colunas")
```

PASSO 3: Tratar valores em branco como nulos

```
df_limpo = df_limpo.replace(['', ' ', 'nan', 'NaN', 'None', 'null'], None)
```

```
print("✅ Passo 3: Valores em branco convertidos para nulos")
```

PASSO 4: Converter tipos de dados

```
# Tentar converter idade para numérico
```

```
df_limpo['idade'] = pd.to_numeric(df_limpo['idade'], errors='coerce')
```

```
# Converter datas
```

```
for fmt in ['%Y-%m-%d', '%Y/%m/%d', '%d-%m-%Y', '%d/%m/%Y']:
```

```
    try:
```

```
        df_limpo['data_compra'] = pd.to_datetime(df_limpo['data_compra'],
format=fmt, errors='coerce')
```

```
        if not df_limpo['data_compra'].isnull().all():
```

```
            break
```

```
    except:
```

```
        continue
```

```
print("✅ Passo 4: Tipos de dados corrigidos")
```

```
# PASSO 5: Validar emails
```

```
import re

def validar_email(email):

    if pd.isna(email):

        return None

    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    return email if re.match(pattern, email) else None

df_limpo['email'] = df_limpo['email'].apply(validar_email)

print("✅ Passo 5: Emails validados")


# PASSO 6: Padronizar telefones

def formatar_telefone(tel):

    if pd.isna(tel) or tel == '':

        return None
```

```

# Remover não-numéricos

numeros = re.sub(r'\D', '', str(tel))


# Verificar se tem número suficiente

if len(numeros) == 11: # Com DDD + 9 dígitos

    return f'({numeros[:2]}) {numeros[2:7]}-{numeros[7:]}'

elif len(numeros) == 10: # Com DDD + 8 dígitos

    return f'({numeros[:2]}) {numeros[2:6]}-{numeros[6:]}'

else:

    return None


df_limpo['telefone'] = df_limpo['telefone'].apply(formatar_telefone)

print("✅ Passo 6: Telefones padronizados")


# PASSO 7: Tratar outliers (valor_compra)

```

```

if 'valor_compra' in df_limpo.columns:

    Q1 = df_limpo['valor_compra'].quantile(0.25)

    Q3 = df_limpo['valor_compra'].quantile(0.75)

    IQR = Q3 - Q1

    limite_inferior = Q1 - 1.5 * IQR

    limite_superior = Q3 + 1.5 * IQR

    # Marcar outliers sem removê-los

    df_limpo['eh_outlier'] = (df_limpo['valor_compra'] < limite_inferior)

| \

                                (df_limpo['valor_compra'] > limite_superior)

    print(f"✅ Passo 7: {df_limpo['eh_outlier'].sum()} outliers
identificados")

# PASSO 8: Preencher valores nulos estrategicamente

if 'valor_compra' in df_limpo.columns:

```

```
media_valor = df_limpo['valor_compra'].mean()

df_limpo['valor_compra'] =
df_limpo['valor_compra'].fillna(media_valor)


if 'idade' in df_limpo.columns:

    mediana_idade = df_limpo['idade'].median()

    df_limpo['idade'] = df_limpo['idade'].fillna(mediana_idade)


print("✅ Passo 8: Valores nulos preenchidos")


# PASSO 9: Remover duplicatas

duplicatas_antes = df_limpo.duplicated().sum()

df_limpo = df_limpo.drop_duplicates()

duplicatas_depois = df_limpo.duplicated().sum()

print(f"✅ Passo 9: {duplicatas_antes - duplicatas_depois} duplicatas
removidas")
```

```
print(f"\n🎯 RESULTADO FINAL: {len(df_limpo)} registros limpos")
```

```
return df_limpo
```

```
# Executando limpeza
```

```
dados_limpos = limpar_dados_completamente(dados_sujos)
```

```
print("\n🧹 DADOS LIMPOS:")
```

```
print(dados_limpos)
```

Técnicas Específicas para Problemas Comuns

1. Lidando com Datas Inconsistentes

```
python
```

```
# 📌 EXEMPLO: Múltiplos formatos de data
```

```
datas_despadronizadas = pd.DataFrame({
```

```
    'id': [1, 2, 3, 4, 5],
```

```
        'data_str': ['01/10/2023', '2023-10-02', '02/10/2023', '2023/10/04',  
                    '05-10-2023']
```

```
    })
```

```
print(f"🔍 17 DATAS EM FORMATOS DIFERENTES:")
```

```
print(datas_despadronizadas)
```

```
# Estratégia: Tentar múltiplos formatos
```

```
formatos = ['%d/%m/%Y', '%Y-%m-%d', '%Y/%m/%d', '%d-%m-%Y']
```

```
datas_despadronizadas['data_corrigida'] = None
```

```
for fmt in formatos:
```

```
    # Aplicar formato atual
```

```
    datas_tentativa = pd.to_datetime(  
  
        datas_despadronizadas['data_str'],  
  
        format=fmt,
```

```

        errors='coerce'

    )

# Preencher apenas os que ainda estão nulos

    mask = datas_despadronizadas['data_corrigida'].isnull() &
datas_tentativa.notnull()

    datas_despadronizadas.loc[mask, 'data_corrigida'] = datas_tentativa[mask]

print("\n✅ DATAS PADRONIZADAS:")

print(datas_despadronizadas[['data_str', 'data_corrigida']])

```

2. Tratamento de Textos "Sujos"

python

📝 EXEMPLO: Limpeza de textos com múltiplos problemas

```

textos_sujos = pd.DataFrame({

    'id': [1, 2, 3, 4],

    'descricao': [

```

```
        ' PRODUTO A - TAMANHO M ',

        'produto b (usado)',

        'PRODUTO C - COR:VERMELHA',

        ' produto d - em promoção!!! '

    ]

}))

print("📝 TEXTOS COM PROBLEMAS:")

print(textos_sujos)

# Pipeline de limpeza de texto

textos_sujos['descricao_limpa'] = textos_sujos['descricao'].str.lower() #
Minúsculas

textos_sujos['descricao_limpa'] = textos_sujos['descricao_limpa'].str.strip()
# Espaços
```

```
textos_sujos['descricao_limpa'] =  
textos_sujos['descricao_limpa'].str.replace(r'[^\\w\\s-]', '', regex=True) #  
Pontuação
```

```
textos_sujos['descricao_limpa'] =  
textos_sujos['descricao_limpa'].str.replace(r'\\s+', ' ', regex=True) #  
Espaços múltiplos
```

```
print("\\n✨ TEXTOS LIMPOS:")
```

```
print(textos_sujos[['descricao', 'descricao_limpa']])
```

3. Identificação e Tratamento de Outliers

python

```
# 📊 EXEMPLO: Detectando valores extremos em vendas
```

```
vendas_com_outliers = pd.DataFrame({  
  
    'vendedor': ['Ana', 'Bruno', 'Carla', 'Diego', 'Elena', 'Fabio'],  
  
    'vendas': [10000, 12000, 11000, 9500, 500000, 10500] # Elena é outlier  
  
})
```

```
print("💰 VENDAS COM OUTLIER:")
```

```
print(vendas_com_outliers)
```

```
# Método IQR (Interquartile Range)
```

```
Q1 = vendas_com_outliers['vendas'].quantile(0.25)
```

```
Q3 = vendas_com_outliers['vendas'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
limite_inferior = Q1 - 1.5 * IQR
```

```
limite_superior = Q3 + 1.5 * IQR
```

```
print(f"\n🔍 DETECÇÃO DE OUTLIERS:")
```

```
print(f"Q1 (25%): R$ {Q1:.2f}")
```

```
print(f"Q3 (75%): R$ {Q3:.2f}")
```

```
print(f"IQR: R$ {IQR:.2f}")
```

```
print(f"Limite inferior: R$ {limite_inferior:.2f}")
```

```
print(f"Limite superior: R$ {limite_superior:.2f}")
```

```
# Identificar outliers
```

```
vendas_com_outliers['eh_outlier'] = (  
  
    (vendas_com_outliers['vendas'] < limite_inferior) |  
  
    (vendas_com_outliers['vendas'] > limite_superior)  
  
)
```

```
print("\n🎯 OUTLIERS IDENTIFICADOS:")
```

```
print(vendas_com_outliers[vendas_com_outliers['eh_outlier']])
```

```
# Estratégias de tratamento
```

```
print("\n🔧 ESTRATÉGIAS DE TRATAMENTO:")
```

```
print("1. Remover: df[~df['eh_outlier']]")
```

```
print("2. Limitar: df['vendas'].clip(lower=limite_inferior,  
upper=limite_superior)")
```

```
print("3. Substituir pela mediana")
```

```
print("4. Investigar (pode ser dado válido!)")
```

4. Padronização de Categorias

python

```
# 📁 EXEMPLO: Categorias inconsistentes
```

```
categorias_despadronizadas = pd.DataFrame({  
  
    'produto': ['Produto A', 'Produto B', 'Produto C', 'Produto D', 'Produto  
E'],  
  
    'categoria': ['ELETRONICO', 'eletrônico', 'Eletrônico', 'ELETRÔNICO',  
'informatica']  
  
})
```

```
print("📁 CATEGORIAS DESPADRONIZADAS:")
```

```
print(categorias_despadronizadas)
```

```
# Dicionário de mapeamento
```

```
mapeamento_categorias = {  
  
    'eletronico': 'Eletrônicos',  
  
    'eletrônico': 'Eletrônicos',  
  
    'elêtronico': 'Eletrônicos',  
  
    'informatica': 'Informática',  
  
    'tecnologia': 'Tecnologia'  
  
}
```

```
# Padronizar primeiro (minúsculas, sem acentos)
```

```
import unicodedata
```

```
def padronizar_texto(texto):
```

```
    if pd.isna(texto):
```

```
        return texto
```

```
# Minúsculas
```

```
texto = texto.lower()
```

```
# Remover acentos
```

```
texto = unicodedata.normalize('NFKD', texto).encode('ASCII',  
'ignore').decode('ASCII')
```

```
return texto
```

```
categorias_despadronizadas['categoria_padronizada'] =  
categorias_despadronizadas['categoria'].apply(padronizar_texto)
```

```
# Aplicar mapeamento
```

```
categorias_despadronizadas['categoria_final'] =  
categorias_despadronizadas['categoria_padronizada'].map(mapeamento_categorias)
```

```
# Se não mapeado, manter original (capitalizada)

categorias_despadronizadas['categoria_final'] =
categorias_despadronizadas['categoria_final'].fillna(

    categorias_despadronizadas['categoria'].str.title()

)

print("\n✅ CATEGORIAS PADRONIZADAS:")

print(categorias_despadronizadas)
```

Caso Prático: Pipeline de Qualidade de Dados

python

```
# 🎯 DESAFIO: Sistema automatizado de qualidade de dados
```

```
class QualityAssurance:
```

```
    """Classe para garantir qualidade de dados"""
```

```
    def __init__(self, df):
```

```
self.df = df.copy()
```

```
self.problemas = []
```

```
self.mtricas = {}
```

```
def verificar_completude(self):
```

```
    """Verifica valores nulos"""
```

```
    nulos = self.df.isnull().sum()
```

```
    total_nulos = nulos.sum()
```

```
    pct_nulos = (total_nulos / (self.df.shape[0] * self.df.shape[1])) *
```

100

```
    self.mtricas['completude'] = {
```

```
        'total_nulos': total_nulos,
```

```
        'pct_nulos': pct_nulos,
```

```
        'nulos_por_coluna': nulos.to_dict()
```

```
    }
```

```
if pct_nulos > 5:
```

```
    self.problemas.append(f"Alta taxa de nulos: {pct_nulos:.1f}%")
```

```
return self
```

```
def verificar_consistencia(self):
```

```
    """Verifica tipos de dados e consistência"""
```

```
    tipos = self.df.dtypes.to_dict()
```

```
    inconsistente = []
```

```
    for col, tipo in tipos.items():
```

```
        if 'object' in str(tipo):
```

```
            # Verificar se coluna object deveria ser numérica
```

```
            try:
```

```
        pd.to_numeric(self.df[col], errors='raise')

        inconsistente.append(f"{col}: Possível número como texto")

    except:

        pass

self.mtricas['consistencia'] = {

    'tipos': tipos,

    'inconsistencias': inconsistente

}

if inconsistente:

    self.problemas.extend(inconsistente)

return self
```

```
def verificar_unicidade(self):

    """Verifica duplicatas e valores únicos"""

    duplicatas = self.df.duplicated().sum()

    pct_duplicatas = (duplicatas / len(self.df)) * 100

    self.mtricas['unicidade'] = {

        'duplicatas': duplicatas,

        'pct_duplicatas': pct_duplicatas

    }

    if pct_duplicatas > 1:

        self.problemas.append(f"Muitas duplicatas: {pct_duplicatas:.1f}%")

    return self
```

```

def gerar_relatorio(self):

    """Gera relatório completo"""

    print("=" * 60)

    print("📊 RELATÓRIO DE QUALIDADE DE DADOS")

    print("=" * 60)


    print(f"\n📈 Dimensões: {self.df.shape[0]} linhas ×  
{self.df.shape[1]} colunas")


    print("\n✅ MÉTRICAS:")

    for categoria, dados in self.metricas.items():

        print(f"\n {categoria.upper()}:")

        for chave, valor in dados.items():

            if isinstance(valor, dict):

                print(f" {chave}:")

                for k, v in valor.items():

```

```
        print(f"        {k}: {v}")

    else:

        print(f"        {chave}: {valor}")

print("\n⚠️  PROBLEMAS IDENTIFICADOS:")

if self.problemas:

    for i, problema in enumerate(self.problemas, 1):

        print(f"    {i}. {problema}")

    else:

        print("    Nenhum problema crítico identificado!")

print(f"\n🎯 SCORE DE QUALIDADE: {self.calcular_score():.1f}/100")

return self
```

```
def calcular_score(self):

    """Calcula score de qualidade (0-100)"""

    score = 100

    # Penalidades

    if self.metricas.get('completude', {}).get('pct_nulos', 0) > 5:

        score -= 20

    if self.metricas.get('unicidade', {}).get('pct_duplicatas', 0) > 1:

        score -= 15

    if self.metricas.get('consistencia', {}).get('inconsistencias', []):

        score -= len(self.metricas['consistencia']['inconsistencias']) * 5

    return max(0, score)
```

```
# Testando o sistema
```

```
qa = QualityAssurance(dados_sujos)
```

```
qa.verificar_completude().verificar_consistencia().verificar_unicidade().gerar  
_relatorio()
```

Checklist de Limpeza para Projetos Reais

```
python
```

```
#  CHECKLIST PRÁTICO PARA TODO PROJETO
```

```
checklist_limpeza = {
```

```
    'Pré-limpeza': [
```

```
        'Fazer backup dos dados originais',
```

```
        'Documentar problemas conhecidos',
```

```
        'Definir critérios de aceitação'
```

```
    ],
```

'Limpeza Básica': [

'Remover espaços em branco desnecessários',

'Converter tipos de dados apropriadamente',

'Tratar valores nulos e em branco',

'Padronizar formatos (data, moeda, etc.)'

],

'Validação': [

'Verificar consistência entre colunas relacionadas',

'Identificar e tratar outliers',

'Validar regras de negócio',

'Remover duplicatas indevidas'

],

'Pós-limpeza': [

```
        'Documentar transformações aplicadas',

        'Validar integridade dos dados',

        'Exportar versão limpa',

        'Criar log de alterações'

    ]

}
```

```
print("🧹 CHECKLIST DE LIMPEZA DE DADOS")
```

```
for categoria, itens in checklist_limpeza.items():
```

```
    print(f"\n📌 {categoria}:")
```

```
        for item in itens:
```

```
            print(f"    ☐ {item}")
```

Dicas Profissionais

1. Nunca modifique dados originais - sempre trabalhe com cópias
2. Documente cada transformação - você vai esquecer
3. Valide após cada etapa - pequenos erros se propagam
4. Use versionamento para seus scripts de limpeza
5. Crie pipelines reproduzíveis - outras pessoas precisam replica

Glossário de Funções

Criação e Leitura

Função	Descrição	Exemplo
<code>pd.DataFrame()</code>	Cria DataFrame a partir de dict/list	<code>df = pd.DataFrame(dados)</code>
<code>pd.read_csv()</code>	Lê arquivo CSV	<code>df = pd.read_csv('arquivo.csv')</code>
<code>pd.read_excel()</code>	Lê arquivo Excel	<code>df = pd.read_excel('arquivo.xlsx')</code>
<code>pd.read_json()</code>	Lê arquivo JSON	<code>df = pd.read_json('arquivo.json')</code>

Inspeção

Função	Descrição	Exemplo
<code>.head(n)</code>	Primeiras n linhas	<code>df.head(5)</code>
<code>.tail(n)</code>	Últimas n linhas	<code>df.tail(3)</code>
<code>.info()</code>	Informações do DataFrame	<code>df.info()</code>

<code>.describe()</code>	Estatísticas descritivas	<code>df.describe()</code>
<code>.shape</code>	Dimensões (linhas, colunas)	<code>df.shape</code>
<code>.columns</code>	Nomes das colunas	<code>df.columns.tolist()</code>
<code>.dtypes</code>	Tipos de dados das colunas	<code>df.dtypes</code>

Seleção

Função	Descrição	Exemplo
<code>df[col]</code>	Seleciona uma coluna	<code>df['nome']</code>
<code>df[[col1, col2]]</code>	Seleciona múltiplas colunas	<code>df[['nome', 'idade']]</code>
<code>.loc[]</code>	Seleção por rótulo	<code>df.loc[0, 'nome']</code>
<code>.iloc[]</code>	Seleção por posição	<code>df.iloc[0, 1]</code>
<code>.query()</code>	Filtro com string	<code>df.query('idade > 30')</code>

Filtragem

Função	Descrição	Exemplo
<code>.isin()</code>	Valores em lista	<code>df[df.cidade.isin(['SP', 'RJ'])]</code>

<code>.str.contains()</code>	String contém padrão	<code>df[df.nome.str.contains('Silva')]</code>
<code>.str.startswith()</code>	Começa com string	<code>df[df.email.str.startswith('a')]</code>
<code>.str.endswith()</code>	Termina com string	<code>df[df.email.str.endswith('.com')]</code>

Agregação

Função	Descrição	Exemplo
<code>.groupby()</code>	Agrupa dados	<code>df.groupby('cidade')</code>
<code>.agg()</code>	Aplica múltiplas agregações	<code>df.agg({'idade': ['mean', 'max']})</code>
<code>.sum()</code>	Soma dos valores	<code>df['valor'].sum()</code>
<code>.mean()</code>	Média	<code>df['idade'].mean()</code>
<code>.median()</code>	Mediana	<code>df['salario'].median()</code>
<code>.min()</code> / <code>.max()</code>	Mínimo/Máximo	<code>df['idade'].min()</code>
<code>.count()</code>	Contagem	<code>df['nome'].count()</code>
<code>.std()</code>	Desvio padrão	<code>df['valor'].std()</code>

<code>.value_counts()</code>	Contagem de valores únicos	<code>df['cidade'].value_counts()</code>
<code>.pivot_table()</code>	Tabela dinâmica	<code>pd.pivot_table(df, values='valor', index='cidade')</code>

Manipulação

Função	Descrição	Exemplo
<code>.drop()</code>	Remove linhas/colunas	<code>df.drop(columns=['col1'])</code>
<code>.rename()</code>	Renomeia colunas	<code>df.rename(columns={'old': 'new'})</code>
<code>.sort_values()</code>	Ordena valores	<code>df.sort_values('idade')</code>
<code>.reset_index()</code>	Reseta índice	<code>df.reset_index(drop=True)</code>
<code>.append()</code>	Adiciona linha	<code>df.append(nova_linha)</code>
<code>.merge()</code>	Junta DataFrames	<code>pd.merge(df1, df2, on='id')</code>
<code>.concat()</code>	Concatena DataFrames	<code>pd.concat([df1, df2])</code>
<code>.apply()</code>	Aplica função	<code>df['idade'].apply(lambda x: x + 1)</code>

<code>.map()</code>	Mapeia valores	<code>df['cidade'].map({'SP': 'São Paulo'})</code>
<code>.astype()</code>	Converte tipo	<code>df['idade'].astype(float)</code>

Limpeza

Função	Descrição	Exemplo
<code>.isnull() / .isna()</code>	Identifica nulos	<code>df.isnull().sum()</code>
<code>.notnull() / .notna()</code>	Identifica não nulos	<code>df[df.nome.notnull()]</code>
<code>.fillna()</code>	Preenche nulos	<code>df.fillna(0)</code>
<code>.dropna()</code>	Remove nulos	<code>df.dropna()</code>
<code>.duplicated()</code>	Identifica duplicatas	<code>df.duplicated().sum()</code>
<code>.drop_duplicates()</code>	Remove duplicatas	<code>df.drop_duplicates()</code>
<code>.replace()</code>	Substitui valores	<code>df.replace({'': 'Não informado'})</code>
<code>.str.strip()</code>	Remove espaços	<code>df['nome'].str.strip()</code>
<code>.str.upper() / .lower()</code>	Maiúsculas/Minúsculas	<code>df['nome'].str.upper()</code>

Exportação

Função	Descrição	Exemplo
<code>.to_csv()</code>	Exporta para CSV	<code>df.to_csv('saida.csv')</code>
<code>.to_excel()</code>	Exporta para Excel	<code>df.to_excel('saida.xlsx')</code>
<code>.to_json()</code>	Exporta para JSON	<code>df.to_json('saida.json')</code>
<code>.to_dict()</code>	Converte para dicionário	<code>df.to_dict('records')</code>

Funções Numéricas

Função	Descrição	Exemplo
<code>.round()</code>	Arredonda números	<code>df['valor'].round(2)</code>
<code>.abs()</code>	Valor absoluto	<code>df['diferenca'].abs()</code>
<code>.cumsum()</code>	Soma cumulativa	<code>df['valor'].cumsum()</code>
<code>.pct_change()</code>	Variação percentual	<code>df['valor'].pct_change()</code>
<code>.clip()</code>	Limita valores	<code>df['valor'].clip(lower=0)</code>

17 Datas e Tempos

Função	Descrição	Exemplo
--------	-----------	---------

<code>pd.to_datetime()</code>	Converte para datetime	<code>pd.to_datetime(df['data'])</code>
<code>.dt.year / .dt.month</code>	Extrai ano/mês	<code>df['data'].dt.year</code>
<code>.dt.strftime()</code>	Formata data	<code>df['data'].dt.strftime('%d/%m/%Y')</code>
<code>.diff()</code>	Diferença entre datas	<code>df['data'].diff()</code>